ÍNDICE

ÍNDICE 1
DISEÑO TÉCNICO 1
BP Internet Banking — Plataforma Digital Bancaria 1
1. Contexto del Sistema 1
2. Objetivos y Alcance 2
3. Restricciones y Supuestos 2
4. Modelo C4 de Arquitectura 3
5. Patrones de Arquitectura 5
6. Flujos Clave del Sistema 7
7. Seguridad en Capas 10
8. Alta Disponibilidad y DR 10
9. Observabilidad 11
10. Infraestructura AWS 12
11. Cumplimiento Normativo 13
12. Estimación de Costos 14
13. Decisiones Arquitectónicas (ADRs) 15
14. Glosario 16

DISEÑO TÉCNICO

BP Internet Banking — Plataforma Digital Bancaria

Campo	Detalle
Documento	DT-BP-IB-2026-001
Versión	1.0
Fecha	Marzo 2026
Autor	Gustavo Paredes
Entidad	Banco BP
Clasificación	Confidencial — Uso Interno

1. Contexto del Sistema

Banco BP requiere una plataforma de banca digital 24/7 para canales web y móvil, cumpliendo regulaciones SBS Ecuador, LOPDP y PCI-DSS v4.0. La solución es una arquitectura de microservicios en AWS EKS integrada con el Core Bancario legado, Keycloak (OAuth2), redes ACH/SWIFT y notificaciones multicanal.

Problema	Solución
Core Bancario legado sin API moderna	Microservicios con Adapter Pattern
Onboarding presencial costoso	Reconocimiento facial AWS Rekognition

Problema	Solución
Autenticación débil	OAuth2 PKCE + MFA + FIDO2/Passkeys
Inconsistencia en transferencias distribuidas	Saga Pattern con orquestación
Falta de trazabilidad regulatoria	Auditoría append-only con Kafka

---

2. Objetivos y Alcance

Objetivos: Describir la arquitectura completa, justificar decisiones tecnológicas, definir modelos C4, establecer controles de seguridad y cumplimiento normativo, y documentar ADRs.

Dentro del alcance: SPA Next.js 14, App Flutter 3, 8 microservicios Spring Boot 3/Java 21, infraestructura AWS (EKS, Aurora, Redis, MSK), onboarding KYC, OAuth2 PKCE, transferencias ACH/SWIFT, auditoría SBS/LOPDP/PCI-DSS.

Fuera del alcance: Core Bancario legado, banca empresarial, módulo de créditos, ATM, pagos QR.

---

3. Restricciones y Supuestos

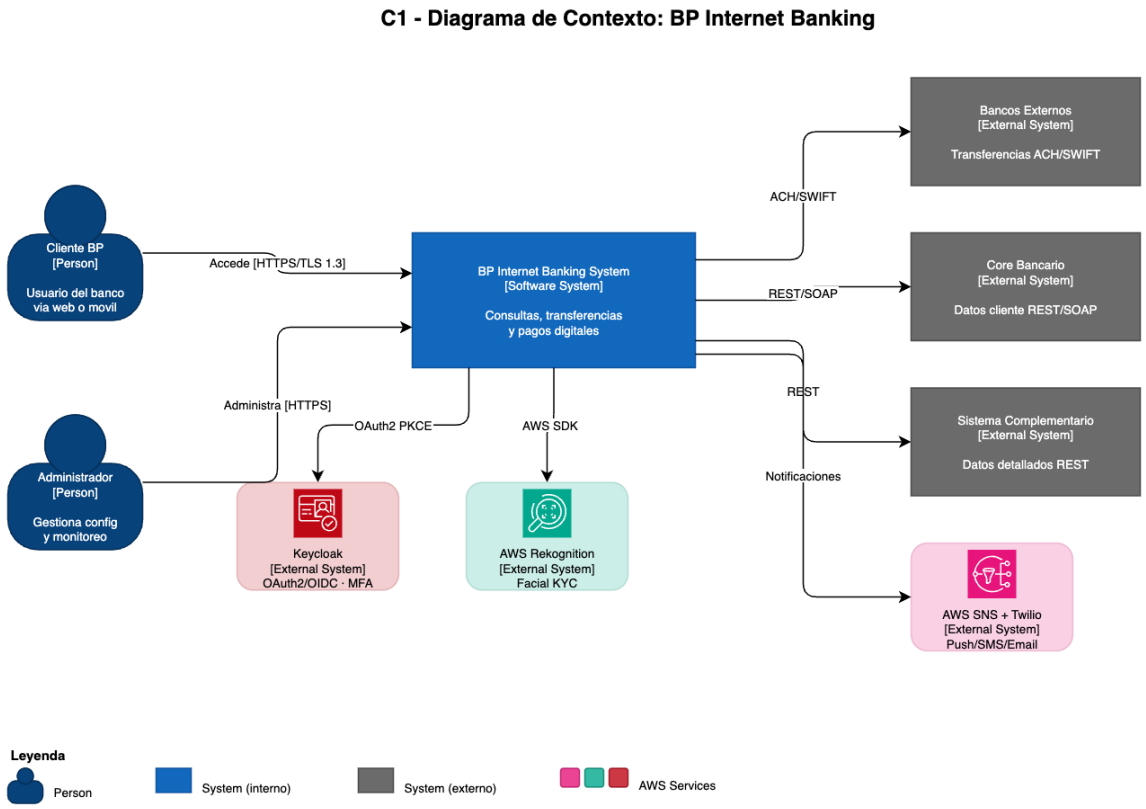
Tipo	Descripción
Regulatoria	SBS Ecuador, LOPDP, PCI-DSS v4.0 obligatorios
Tecnológica	Core Bancario expone REST/SOAP — no modificable
Seguridad	TLS 1.3 mínimo; MFA obligatorio en transferencias
Disponibilidad	SLA 99.95% mensual exigido por SBS
Datos	Sin almacenamiento de CVV; logs auditoría 5 años mínimo

Supuestos: AWS como proveedor único; Core Bancario SLA >= 99.9%; Keycloak en EKS; certificados TLS via ACM.

---

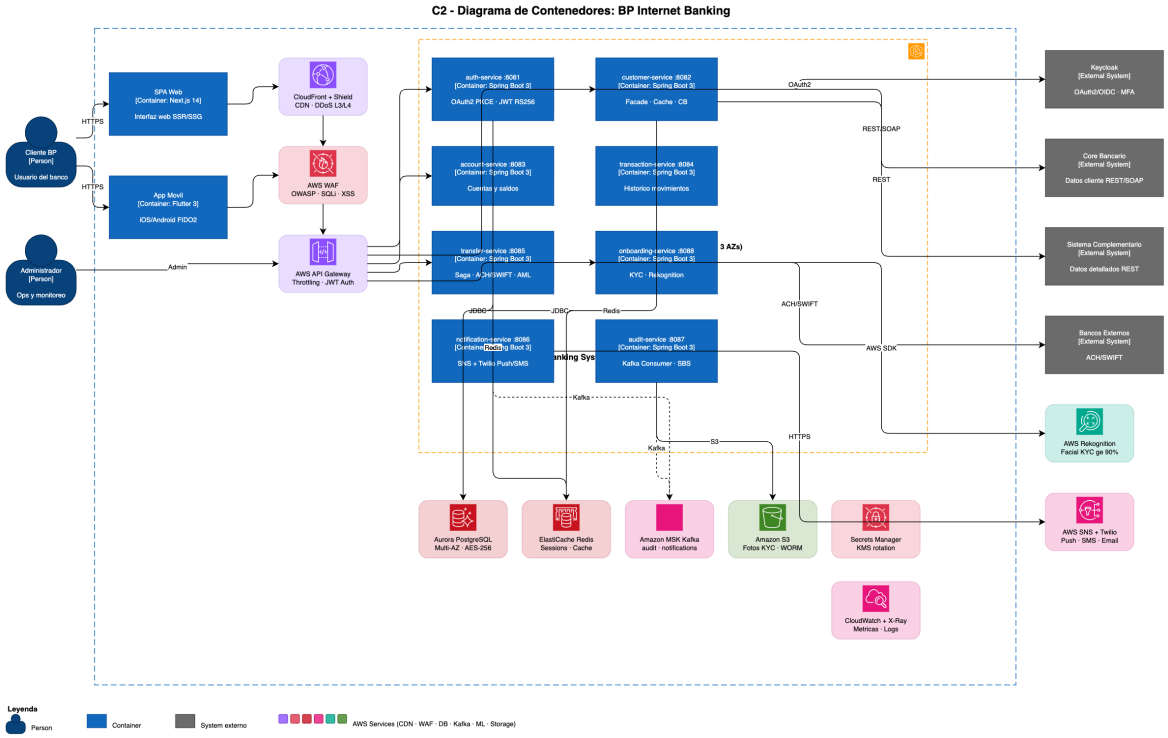
4. Modelo C4 de Arquitectura

4.1 C1 — Contexto



Actor / Sistema	Tipo	Rol
Cliente BP	Persona	Accede vía web o app móvil
Administrador	Persona	Monitoreo y configuración
Core Bancario	Sistema Externo	Cuentas, movimientos, productos (REST/SOAP)
Sistema Complementario	Sistema Externo	Enriquecimiento de datos cliente (REST)
Keycloak	Sistema Externo	OAuth2/OIDC, MFA
AWS Rekognition	Sistema Externo	Reconocimiento facial KYC
Bancos Externos	Sistema Externo	Transferencias ACH/SWIFT
AWS SNS / Twilio	Sistema Externo	Notificaciones email/SMS/push

4.2 C2 — Contenedores



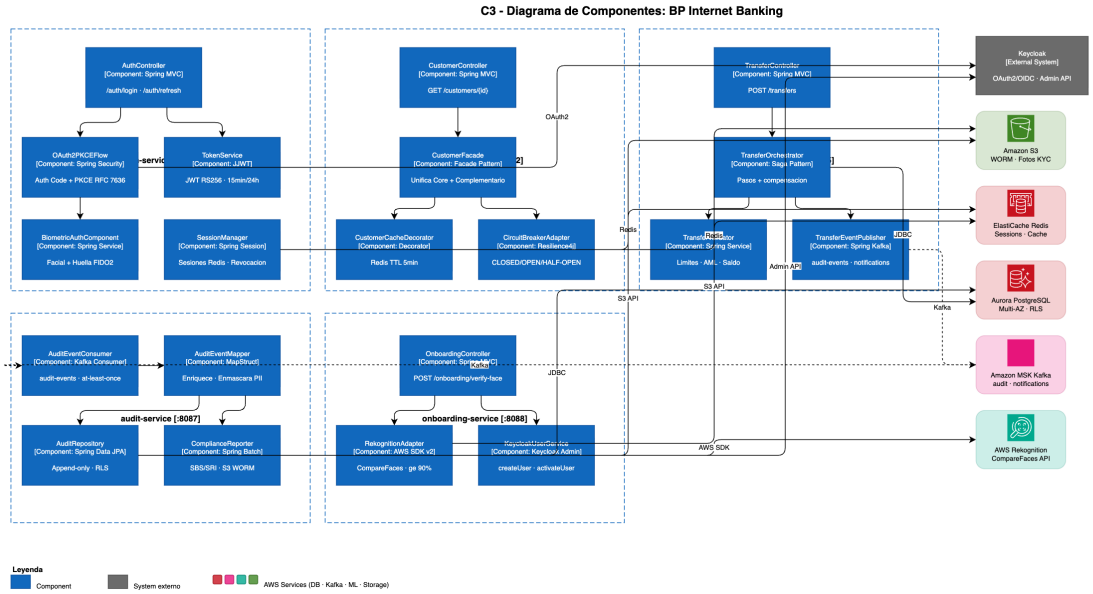
Frontend: SPA Next.js 14 (SSR/SSG, AWS Amplify) + App Flutter 3 (iOS/Android, FIDO2).

Microservicios (Spring Boot 3 / Java 21):

Servicio	Puerto	Responsabilidad
auth-service	8081	OAuth2 PKCE, JWT RS256, biometría
customer-service	8082	Facade + Cache-Aside + Circuit Breaker
account-service	8083	Cuentas y saldos
transaction-service	8084	Histórico de movimientos
transfer-service	8085	Saga + ACH/SWIFT + AML
notification-service	8086	SNS + Twilio
audit-service	8087	Kafka consumer, append-only, SBS
onboarding-service	8088	KYC Rekognition + Keycloak provisioning

Infraestructura AWS: API Gateway (entrada única) → EKS → Aurora PostgreSQL Multi-AZ → ElastiCache Redis → MSK Kafka → Secrets Manager → CloudWatch/X-Ray.

4.3 C3 — Componentes



auth-service: AuthController → OAuth2PKCEFlow + TokenService (JWT RS256) + BiometricAuthComponent (Rekognition/FIDO2) + SessionManager (Redis).

customer-service: CustomerController → CustomerFacade (CoreBankingAdapter + ComplementaryAdapter) → CustomerCacheDecorator (Redis TTL 5min) + CircuitBreakerAdapter (Resilience4j).

transfer-service: TransferController → TransferOrchestrator (Saga) → TransferValidator (AML) + TransferEventPublisher (Kafka).

audit-service: AuditEventConsumer (Kafka) → AuditEventMapper (enmascara PII) → AuditRepository (PostgreSQL append-only, RLS) + ComplianceReporter (S3 WORM).

---

5. Patrones de Arquitectura

Microservicios

Servicios: Todos

- Escalado y despliegue independiente por dominio funcional.
- Aislamiento de fallos: un servicio caído no colapsa el sistema.

Saga (Orquestación)

Servicio: transfer-service

- Evita bloqueos distribuidos del protocolo 2PC en producción bancaria.
- Compensaciones explícitas por paso garantizan consistencia eventual.

Facade

Servicio: customer-service

- Unifica Core Bancario (SOAP) y Sistema Complementario (REST) en una sola API.

- Permite cambiar sistemas subyacentes sin impactar consumidores.

Cache-Aside + Decorator

Servicio: customer-service

- Reduce latencia de ~200 ms (Core) a ~5 ms (Redis), mejorando UX.
- Protege al Core de sobrecarga en picos mediante TTL de 5 minutos.

Circuit Breaker

Servicios: customer-service, transfer-service

- Previene fallos en cascada ante degradación del Core o ACH/SWIFT.
- Habilita respuestas con caché stale manteniendo disponibilidad parcial.

Event-Driven

Servicios: audit-service, notification-service

- Desacopla auditoría y notificaciones del camino crítico transaccional.
- Kafka garantiza entrega at-least-once con retención para replay.

BFF (Backend for Frontend)

Servicio: bff-service

- Agrega múltiples servicios en una sola respuesta, reduciendo round-trips.
- Adapta el contrato de API por canal (SPA vs Flutter) evitando over-fetching.

Adapter

Servicios: Todos

- Aísla interfaces externas heterogéneas (SOAP, ISO 20022, AWS SDK).
- Permite sustituir proveedores sin modificar la lógica de negocio.

Circuit Breaker config (Resilience4j): 5 fallos en 10s → OPEN (30s) → HALF-OPEN (1 request prueba) → fallback Redis.

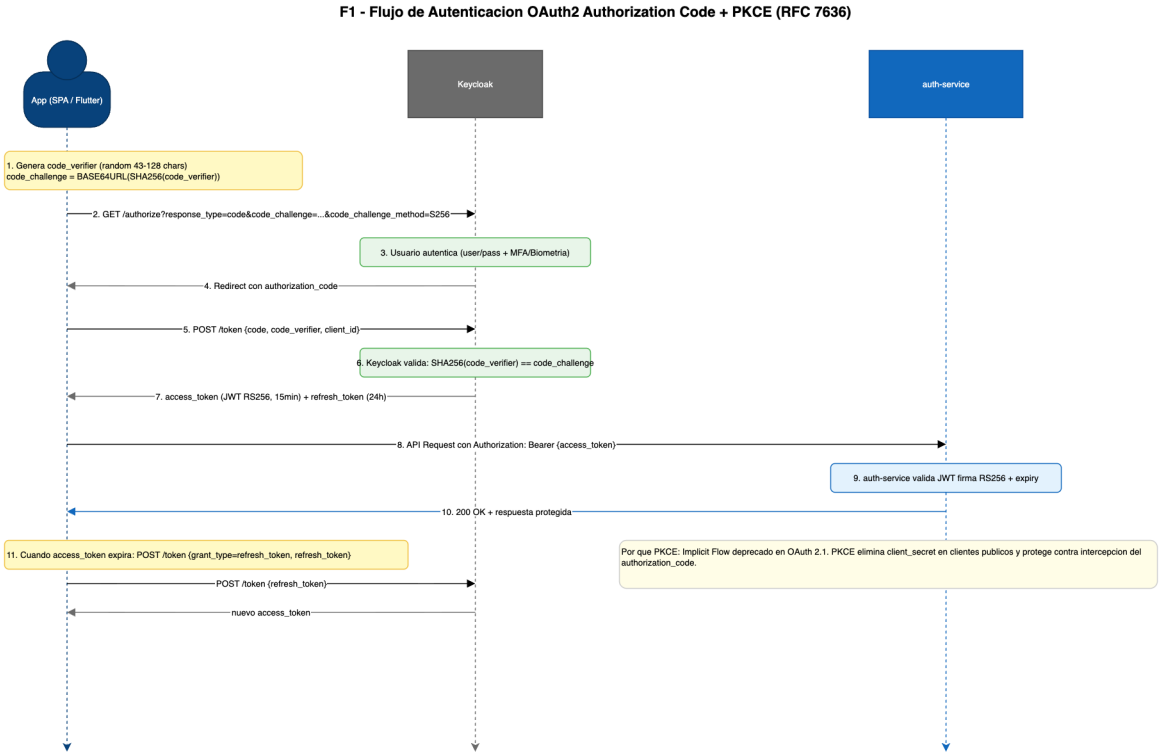
Kafka topics:

Topic	Productor	Consumidor
audit-events	Todos los servicios	audit-service
transfer-events	transfer-service	notification-service, audit-service
onboarding-events	onboarding-service	notification-service, audit-service

---

6. Flujos Clave del Sistema

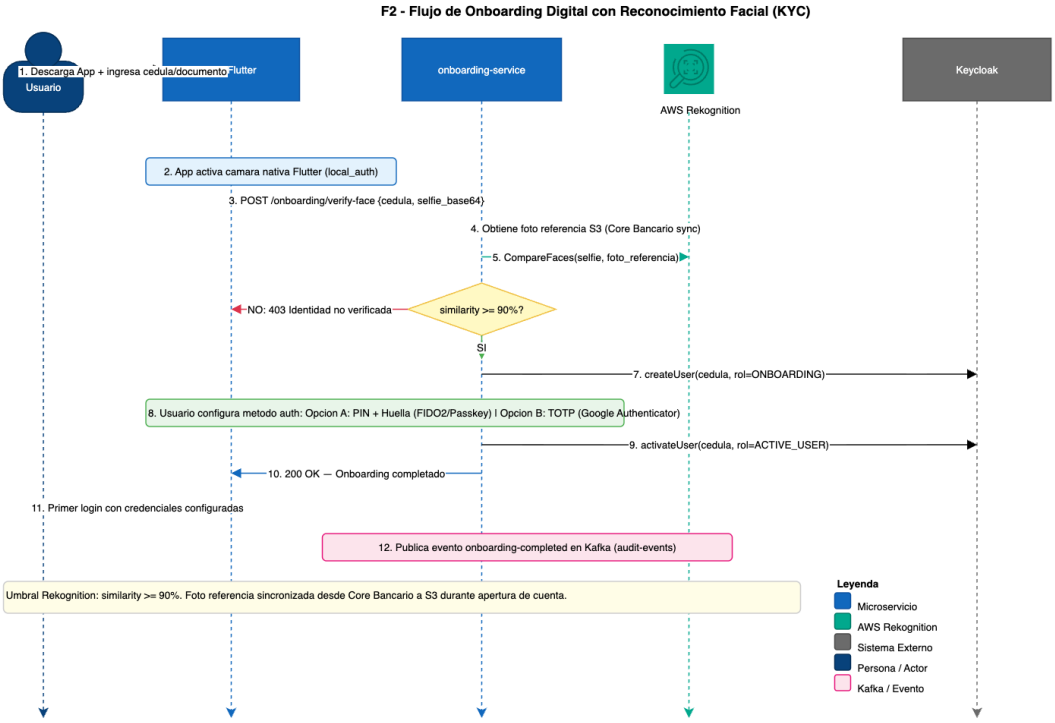
6.1 Autenticación OAuth2 PKCE



1. App genera code_verifier + code_challenge = BASE64URL(SHA256(code_verifier))
2. Redirect a Keycloak con code_challenge
3. Usuario autentica (user/pass + MFA/biometría)
4. Keycloak retorna authorization_code
5. App intercambia code + code_verifier → access_token (JWT 15min) + refresh_token (24h)
6. Requests con Authorization: Bearer {token} validados por auth-service

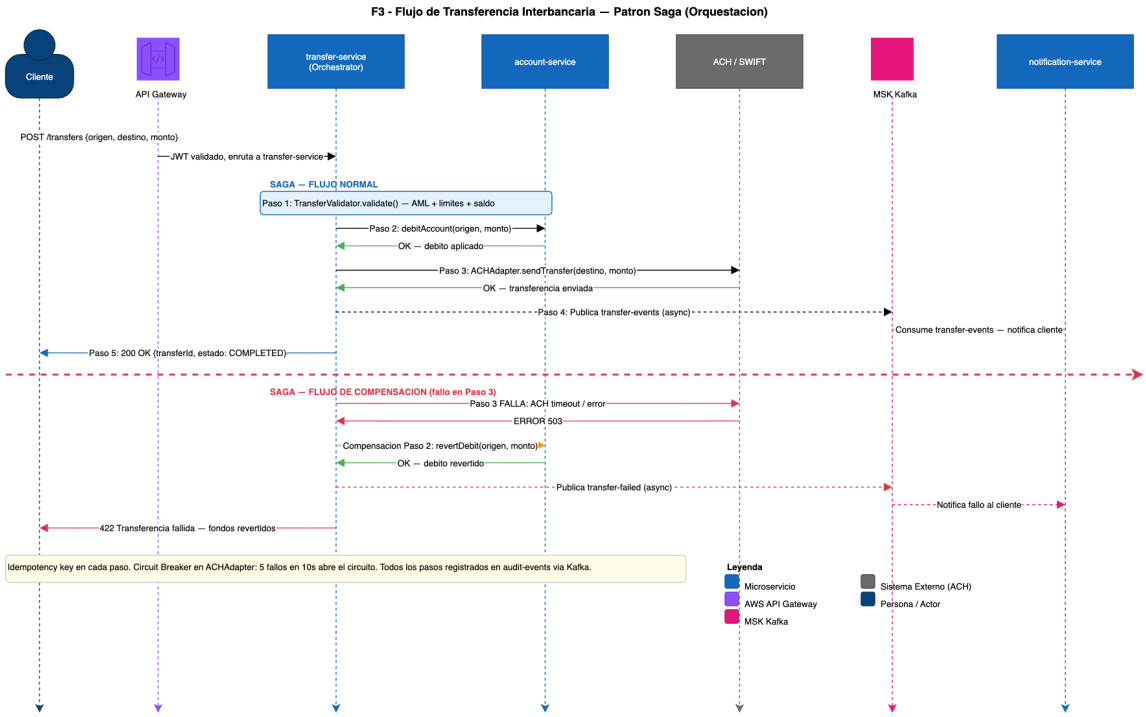
Por qué PKCE: Implicit Flow deprecado en OAuth 2.1. Elimina client_secret en clientes públicos y protege contra intercepción del authorization_code.

6.2 Onboarding Digital con Reconocimiento Facial



1. Usuario ingresa cédula → App captura selfie (cámara nativa Flutter)
2. onboarding-service → RekognitionAdapter.compareFaces(selfie, foto_S3) — umbral >= 90%
3. Si válido → KeycloakUserService.createUser() con rol ONBOARDING
4. Usuario configura: PIN + Huella (FIDO2) o TOTP (Google Authenticator)
5. KeycloakUserService.activateUser() → rol ACTIVE_USER
6. Publica onboarding-completed en Kafka

6.3 Transferencia Interbancaria (Saga)

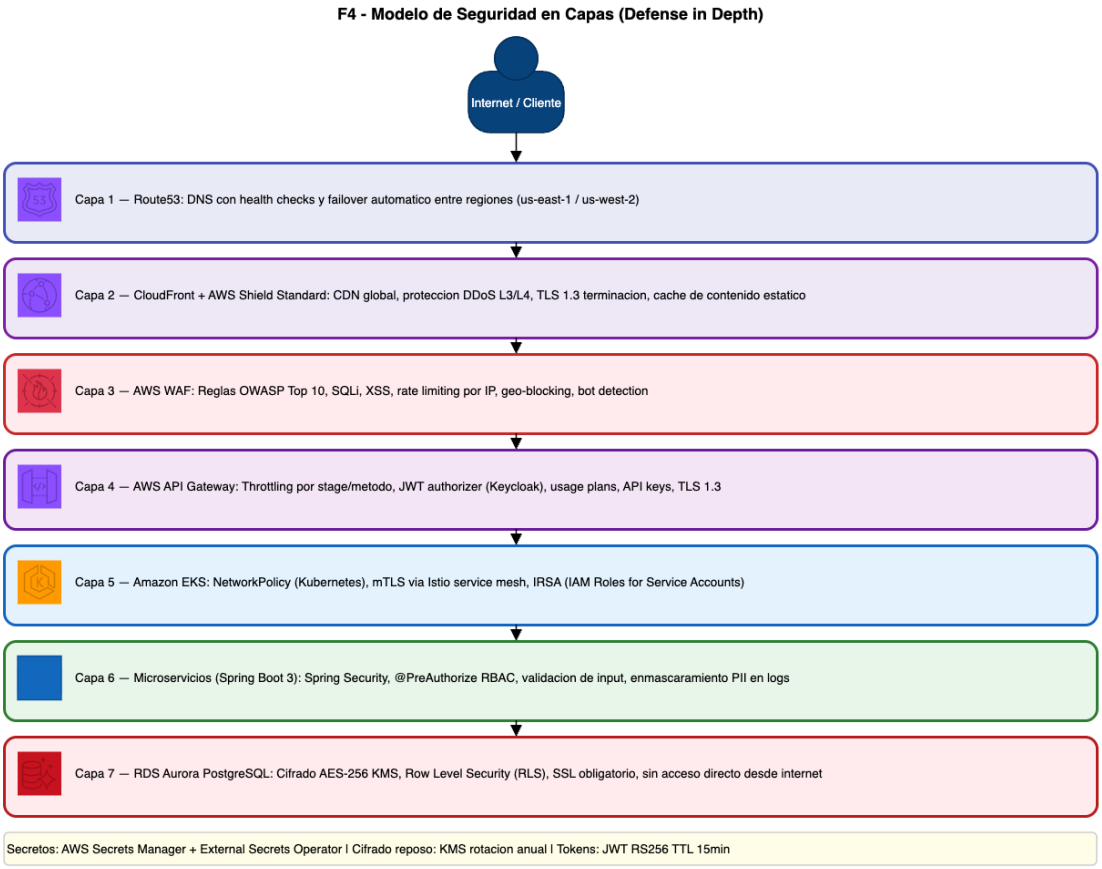


Paso	Acción	Compensación
1	TransferValidator: AML + límites + saldo	LiberarReserva
2	AccountService.debitAccount()	RevertirDebito
3	ACHAdapter.sendTransfer()	RevertirCredito
4	Publica transfer-events (Kafka async)	— idempotente
5	Responde {transferId, COMPLETED}	—

Fallo en Paso 3 → compensaciones 2→1 automáticas → notifica fallo al cliente.

---

7. Seguridad en Capas



Capa	Servicio	Control
1	Route53	DNS con health checks y failover automático
2	CloudFront + Shield	DDoS L3/L4, TLS 1.3 terminación
3	AWS WAF	OWASP Top 10, SQLi, XSS, rate limiting por IP
4	AWS API Gateway	Throttling, JWT authorizer, usage plans, TLS 1.3
5	Amazon EKS	NetworkPolicy, mTLS (Istio), IRSA
6	Microservicios	Spring Security, RBAC (@PreAuthorize), validación input
7	RDS Aurora	AES-256 KMS, RLS, SSL obligatorio

Secretos: AWS Secrets Manager + External Secrets Operator. **Tokens:** JWT RS256, TTL 15min / refresh 24h.

---

8. Alta Disponibilidad y DR

Topología: EKS Multi-AZ (us-east-1: 3 AZs × 5 nodos). DR Warm Standby en us-west-2.

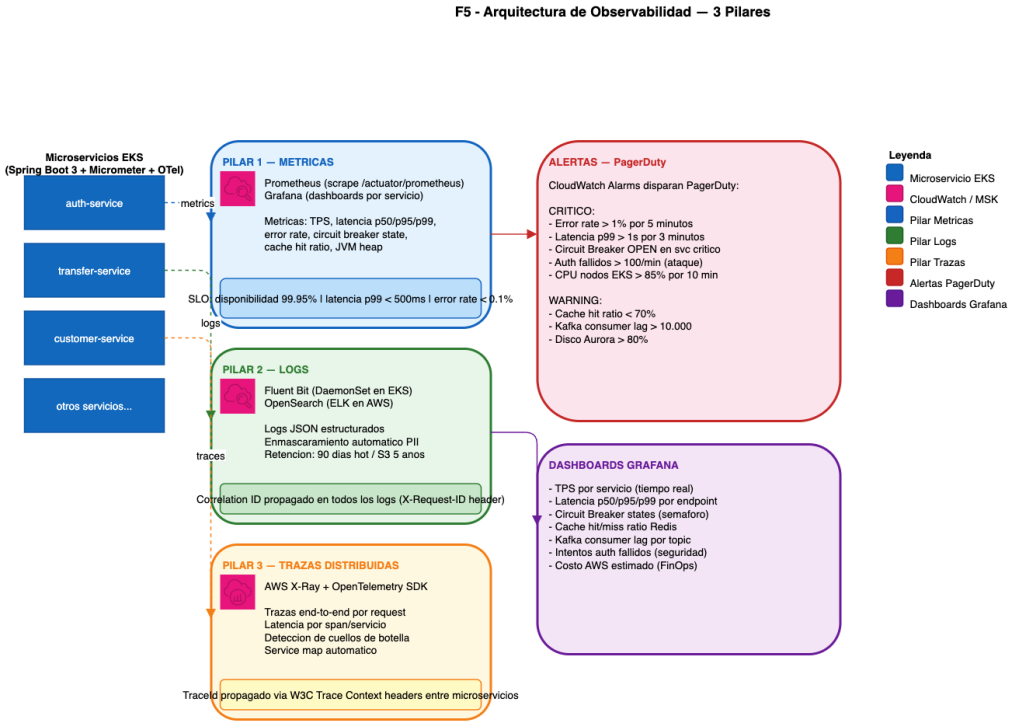
Métrica	Objetivo
RPO	1 hora
RTO	4 horas
Disponibilidad	99.95% mensual

Replicación DR: Aurora Global Database (< 1s) + MSK Mirror Maker 2 + Route53 Failover automático.

Auto-healing: Liveness Probes → reinicio pods | HPA (CPU > 70%) | Cluster Autoscaler + Karpenter | PodDisruptionBudget (min 2 réplicas).

---

9. Observabilidad



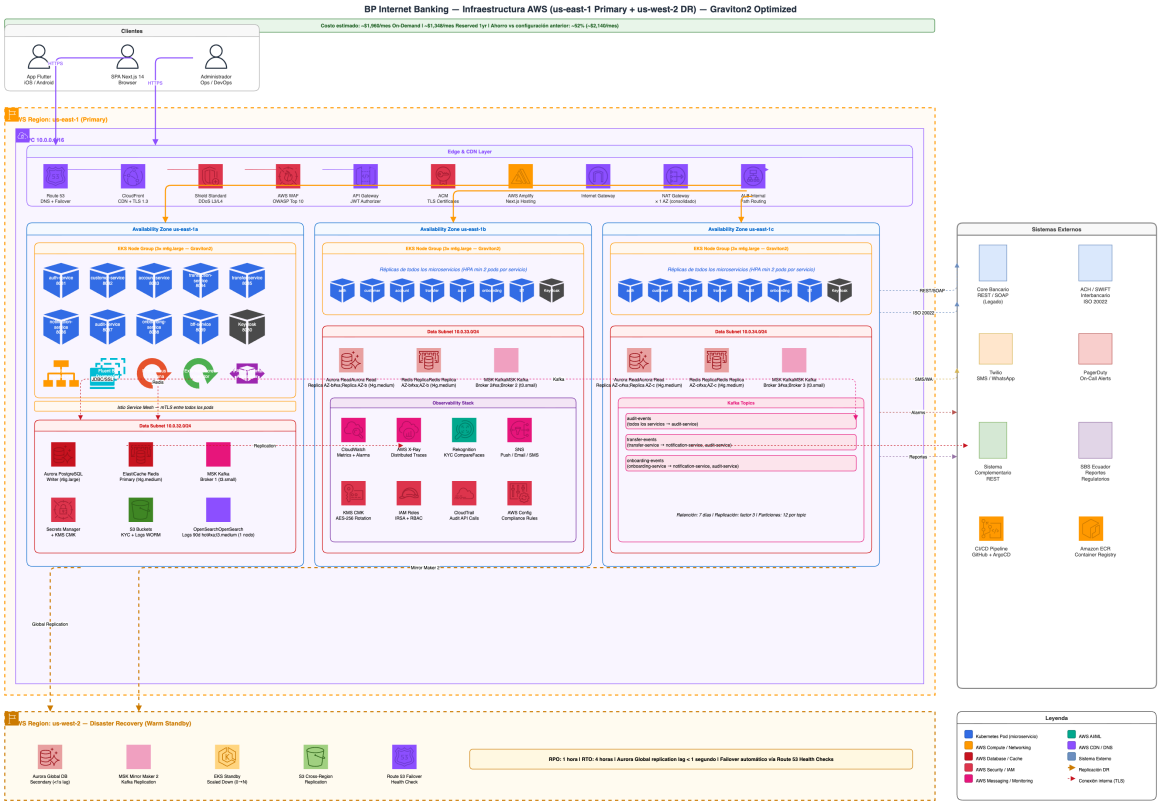
Pilar	Herramienta	Métricas clave
Métricas	Prometheus + Grafana	TPS, latencia p99, error rate, circuit breaker state
Logs	Fluent Bit → OpenSearch	JSON estructurado, enmascaramiento PII, retención 5 años
Trazas	AWS X-Ray + OpenTelemetry	Trazas end-to-end, correlation ID, service map

SLOs: Disponibilidad 99.95% | Latencia p99 consultas < 500ms | Latencia p99 transferencias < 2s | Error rate < 0.1%.

Alertas PagerDuty: Error rate > 1% (5min) | Latencia p99 > 1s (3min) | Circuit Breaker OPEN | Auth fallidos > 100/min | CPU EKS > 85% (10min).

---

10. Infraestructura AWS



La infraestructura despliega todos los componentes en la región primaria **us-east-1** distribuidos en tres Availability Zones, con DR Warm Standby en **us-west-2**.

Capas de red

Capa	Rango CIDR	Componentes
Public / Edge	10.0.0.0/20	Route53, CloudFront, Shield, WAF, API Gateway, ALB, NAT GW, IGW
Private / EKS	10.0.16.0/20	EKS Node Groups (9× m5.xlarge), Istio, Fluent Bit, Prometheus
Data	10.0.32.0/20	Aurora PostgreSQL, ElastiCache Redis, MSK Kafka, Secrets Manager, S3, OpenSearch

Compute — Amazon EKS

- Cluster Multi-AZ: 3 node groups × 3 nodos **m6g.large** (Graviton2, -40% vs m5.xlarge, 9 nodos totales).
- Karpenter gestiona nodos Spot para workloads no críticos (audit, notification) con ahorro adicional ~30%.
- Istio Service Mesh con mTLS entre todos los pods.
- HPA (CPU > 70%) + Karpenter Autoscaler + PodDisruptionBudget (mín. 2 réplicas por servicio).
- IRSA (IAM Roles for Service Accounts) para acceso granular a servicios AWS sin credenciales estáticas.

Datos y mensajería

Servicio	Configuración optimizada	Propósito
Aurora PostgreSQL	Writer **r6g.large** + 1 Reader **t4g.medium** (Multi-AZ)	Transaccional, RLS, AES-256 KMS; reader para consultas de solo lectura
ElastiCache Redis	2 nodos **cache.t4g.medium** (Graviton2, burstable)	Cache-Aside, sesiones JWT, TTL 5 min; eliminado 3er nodo redundante
Amazon MSK	3 brokers **kafka.t3.small** (volumen inicial bancario)	Bus de eventos; escalar a m5.large cuando TPS > 5.000
S3	Object Lock WORM + **Intelligent-Tiering**	Fotos KYC, logs auditoría 5 años; tiering automático reduce costo ~60%
OpenSearch	1 nodo **t3.medium.search** + S3 cold 5 años	Logs estructurados JSON; escalar horizontalmente si volumen crece

Seguridad de infraestructura

- **Secrets Manager + KMS CMK:** rotación anual automática, External Secrets Operator sincroniza secretos a Kubernetes.
- **CloudTrail + AWS Config:** auditoría de todas las llamadas API y evaluación continua de compliance.
- **IAM + IRSA:** principio de mínimo privilegio; sin credenciales de larga duración en pods.
- **VPC Flow Logs + GuardDuty:** detección de anomalías de red en tiempo real.

CI/CD y registro de contenedores

- Pipeline: GitHub Actions → build → test → push a **Amazon ECR** → ArgoCD despliega en EKS (GitOps).
- Imágenes escaneadas con ECR Image Scanning (Trivy) antes de cada despliegue.

Disaster Recovery

Componente	Mecanismo	Latencia
Aurora PostgreSQL	Aurora Global Database	< 1 segundo
MSK Kafka	Mirror Maker 2	< 30 segundos
S3	Cross-Region Replication	Eventual
EKS	Standby escalado a 0, scale-up en failover	RTO 4 horas
DNS	Route53 Health Checks + Failover routing	< 60 segundos

---

11. Cumplimiento Normativo

Norma	Implementación clave
LOPDP (Ecuador)	Consentimiento explícito, derecho al olvido, enmascaramiento PII en logs
SBS Ecuador	MFA obligatorio, TLS 1.3, logs auditoría 5 años, reportes mensuales

Norma	Implementación clave
PCI-DSS v4.0	Tokenización tarjetas, sin CVV, AES-256, segmentación de red
ISO 27001	SGSI implementado, gestión de riesgos, revisión anual
SWIFT CSP	Controles mensajería SWIFT, autenticación operadores
FATF/GAFI (AML)	Motor AML en TransferValidator, reporte operaciones sospechosas
OWASP ASVS L2	Verificación seguridad APIs, pentesting anual

Controles técnicos: PostgreSQL append-only (no_update_audit, no_delete_audit) + S3 Object Lock WORM (5 años) + RLS por cliente + reportes SBS/SRI automáticos mensuales.

---

12. Estimación de Costos

Configuración optimizada (Graviton2 + right-sizing)

Servicio	Configuración	USD/mes On-Demand	USD/mes Reserved 1yr
Amazon EKS	9 nodos **m6g.large** (Graviton2)	~$780	~$468
RDS Aurora PostgreSQL	Writer **r6g.large** + 1 Reader **t4g.medium**	~$320	~$192
ElastiCache Redis	2 nodos **cache.t4g.medium** (Graviton2)	~$100	~$60
Amazon MSK (Kafka)	3 brokers **kafka.t3.small**	~$150	~$90
CloudFront	1 TB/mes	~$85	~$85
AWS API Gateway	10M req/mes	~$35	~$35
AWS WAF	10M req/mes	~$100	~$100
AWS Rekognition	100K faces/mes	~$100	~$100
CloudWatch + X-Ray + OpenSearch	1 nodo t3.medium.search	~$180	~$108
ALB + NAT Gateway (×1) + SM + KMS	NAT consolidado a 1 AZ	~$110	~$110
Total On-Demand		**~$1,960**	
Total Reserved (1 año, -40%)			**~$1,348**

Comparativa vs configuración anterior

Componente	Antes	Después	Ahorro/mes
EKS (9 nodos)	m5.xlarge	m6g.large ~$780	**~$1,020**

Componente	Antes	Después	Ahorro/mes
	~$1,800		
Aurora	r6g.xlarge ×3 ~$600	r6g.large + t4g.medium ~$320	**~$280**
Redis	r6g.large ×3 ~$400	t4g.medium ×2 ~$100	**~$300**
MSK	m5.large ×3 ~$500	t3.small ×3 ~$150	**~$350**
OpenSearch + CW	~$300	~$180	**~$120**
NAT Gateway	×3 ~$90	×1 ~$30	**~$60**
Total ahorro	**~$4,100**	**~$1,960**	**~$2,140 (52%)**

Estrategia de optimización adicional

- **Karpenter Spot** para pods no críticos (audit-service, notification-service): ahorro adicional ~30% en compute.
- **S3 Intelligent-Tiering** para logs históricos: reduce costo de almacenamiento ~60% tras 30 días.
- **Reserved Instances 1 año** en Aurora + Redis + EKS: -40% sobre On-Demand.
- **Savings Plans Compute** (EKS): aplica automáticamente a cualquier instancia Graviton, x86 o Fargate.
- **Escalado a producción:** cuando TPS > 5.000, migrar MSK a m5.large y Redis a r6g.large (~$650/mes adicional).

---

13. Decisiones Arquitectónicas (ADRs)

ADR-01 — Flutter 3 para desarrollo móvil

Alternativa descartada: React Native

- Acceso nativo a hardware biométrico (FIDO2) con mejor rendimiento que el bridge JS de React Native.
- Un codebase para iOS y Android reduce costos de mantenimiento a la mitad.

ADR-02 — Keycloak desplegado en EKS

Alternativa descartada: Auth0, AWS Cognito

- Control total sobre datos de identidad, garantizando cumplimiento LOPDP en infraestructura propia.
- Sin costo por MAU, determinante para una base de clientes bancaria de alto volumen.

ADR-03 — Amazon MSK (Kafka) como bus de eventos

Alternativa descartada: SQS/SNS, RabbitMQ

- Retención configurable permite replay completo ante fallos de consumidores.
- Mayor throughput y particionado garantizan orden estricto por cuenta/cliente.

ADR-04 — AWS API Gateway como punto de entrada

Alternativa descartada: Kong API Gateway

- Integración nativa con IAM, CloudWatch y JWT authorizer para Keycloak sin plugins adicionales.
- Servicio gestionado elimina overhead operacional de mantener Kong en EKS.

ADR-05 — Saga Pattern para transacciones distribuidas

Alternativa descartada: Two-Phase Commit (2PC)

- 2PC genera bloqueos distribuidos que degradan el throughput en producción bancaria.
- Saga provee compensaciones explícitas y trazabilidad completa del estado de cada transacción.

ADR-06 — Aurora PostgreSQL Multi-AZ

Alternativa descartada: DynamoDB

- Transacciones ACID estrictas requeridas para operaciones bancarias; DynamoDB no las garantiza nativamente.
- Replicación Aurora Global < 1 s habilita DR en us-west-2 con RPO de 1 hora.

ADR-07 — OAuth2 con PKCE (RFC 7636)

Alternativa descartada: Implicit Flow

- Implicit Flow está deprecado en OAuth 2.1 y expone tokens en la URL, vulnerable a leakage.
- PKCE elimina la necesidad de client_secret en clientes públicos (SPA/Mobile) sin reducir seguridad.

---

14. Glosario

Término	Definición
ACH	Automated Clearing House — red de pagos interbancarios
ADR	Architecture Decision Record
AML	Anti-Money Laundering — prevención lavado de activos
BFF	Backend for Frontend
FIDO2	Fast Identity Online 2 — autenticación sin contraseña
HPA	Horizontal Pod Autoscaler
IRSA	IAM Roles for Service Accounts
JWT	JSON Web Token
LOPDP	Ley Orgánica de Protección de Datos Personales (Ecuador)
MSK	Amazon Managed Streaming for Apache Kafka
mTLS	Mutual TLS
PKCE	Proof Key for Code Exchange (RFC 7636)
RLS	Row Level Security (PostgreSQL)
RPO / RTO	Recovery Point / Time Objective
Saga	Patrón de transacciones distribuidas con compensaciones
SBS	Superintendencia de Bancos del Ecuador
WORM	Write Once Read Many